

Capacitated Vehicle Routing Problem

Soluzione tramite Algoritmo Genetico Ibrido (HGA)

Giuseppe Bellamacina – Matricola 1000030349

Corso di Artificial and Swarm Intelligence
Prof. Mario Francesco Pavone
Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Anno Accademico 2025/2026

1 Introduzione al Problema

Repository <https://github.com/GiuseppeBellamacina/capacitated-vehicle-routing-problem>

Il **Capacitated Vehicle Routing Problem (CVRP)** è una delle varianti più studiate del classico Vehicle Routing Problem (VRP), introdotto da [1]. Si tratta di un problema di ottimizzazione combinatoria NP-hard [2] che consiste nel determinare un insieme di percorsi di costo minimo per una flotta di veicoli, al fine di servire un dato insieme di clienti geograficamente distribuiti, partendo e ritornando ad un deposito centrale.

Formalmente, sia dato un grafo completo $G = (V, E)$, dove:

- $V = \{v_0, v_1, \dots, v_n\}$ è l'insieme dei vertici, con v_0 che rappresenta il *depot* e $V \setminus \{v_0\}$ l'insieme dei clienti;
- E è l'insieme degli archi, ad ognuno dei quali è associato un costo $d_{ij} \geq 0$ (distanza euclidea tra i nodi i e j).

Ad ogni cliente $i \in V \setminus \{v_0\}$ è associata una domanda $q_i \geq 0$, e sono disponibili m veicoli, ciascuno con capacità σ . L'obiettivo è determinare per ogni veicolo un itinerario tale che:

- (i) Ogni cliente sia visitato esattamente una volta da un solo veicolo;
- (ii) Ogni percorso inizi e termini al depot;
- (iii) La somma delle domande dei clienti assegnati a ciascun veicolo non superi la capacità σ .

L'obiettivo è minimizzare il costo totale:

$$\min \sum_{h=1}^m \sum_{(i,j) \in \eta_h} d_{ij} \quad (1)$$

dove η_h è l'insieme degli archi percorsi dal veicolo h .

2 Rilevanza e Applicazioni

Il CVRP ha un'enorme rilevanza pratica: ottimizzare le rotte di consegna permette alle aziende di logistica di ridurre i costi operativi, il consumo di carburante e le emissioni di CO₂. Le applicazioni spaziano dalla distribuzione di merci alla raccolta rifiuti, dal trasporto scolastico alla consegna dell'ultimo miglio.

La natura NP-hard del problema rende proibitiva la risoluzione esatta per istanze di dimensioni realistiche ($n > 50$ clienti). Per questo motivo, la ricerca si è concentrata su metodi euristici e meta-euristici [3, 4, 5] in grado di produrre soluzioni di alta qualità in tempi computazionali accettabili.

3 Algoritmo Genetico Ibrido (HGA)

Tra gli algoritmi proposti, la scelta è ricaduta sull'**Algoritmo Genetico Ibrido (HGA)** [6, 3], ispirato al framework HGSADC [5]. L'ibridazione con operatori di ricerca locale (2-opt, Or-opt, Relocate, Exchange) [7, 8] combina esplorazione globale e sfruttamento locale.

3.1 Encoding e Algoritmo di Split

L'algoritmo utilizza un **encoding a permutazione** dei clienti. Ogni cromosoma è una permutazione $\pi = (\pi_1, \dots, \pi_n)$ decodificata in un insieme ottimale di route tramite l'**algoritmo di Split** di Prins [4] (DP con complessità $O(n^2)$):

$$V[i] = \min_{j: \sum_{k=j}^i q_{\pi_k} \leq \sigma} \left[V[j-1] + d_{0, \pi_j} + \sum_{k=j}^{i-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_i, 0} \right]$$

Algoritmo 1: Split Algorithm (Prins, 2004)

Input: Permutazione π , matrice distanze d , domande q , capacità Q , depot 0
Output: Rotte ottimali $\mathcal{R}$ e costo totale

```
1 cum[0]  $\leftarrow$  0;  
2 for  $i \leftarrow 1$  to  $n - 1$  do  
3    $cum[i + 1] \leftarrow cum[i] + d[\pi_i, \pi_{i+1}]$ ;  
4  $V[0] \leftarrow 0$ ;  $pred[0] \leftarrow -1$ ;  
5 for  $i \leftarrow 1$  to  $n$  do  
6    $V[i] \leftarrow +\infty$ ;  $load \leftarrow 0$ ;  
7   for  $j \leftarrow i$  to 1 do  
8      $load \leftarrow load + q[\pi_j]$ ;  
9     if  $load > Q$  then  
10      break;  
11      $rc \leftarrow d[0, \pi_j] + (cum[i] - cum[j]) + d[\pi_i, 0]$ ;  
12     if  $V[j - 1] + rc < V[i]$  then  
13        $V[i] \leftarrow V[j - 1] + rc$ ;  $pred[i] \leftarrow j - 1$ ;  
14 return BACKTRACK( $pred, \pi, n$ ),  $V[n]$ ;
```

3.2 Popolazione Iniziale e Operatori Genetici

La popolazione iniziale ($\mu = 81$) combina: Nearest Neighbor Heuristic, Clarke-Wright Savings [9] e permutazioni casuali per garantire diversità genetica. La **selezione a torneo** [10] ($k = 4$) seleziona il migliore tra 4 individui casuali.

Il **crossover Order Crossover (OX)** [11] ($p_c = 0.675$) preserva l'ordine relativo: seleziona due punti di taglio casuali, copia il segmento centrale da P_1 , e riempie le posizioni restanti con i geni di P_2 in ordine, saltando i duplicati (Algoritmo 2).

Algoritmo 2: Order Crossover (OX) [11]

Input: Permutazioni P_1, P_2 di lunghezza n
Output: Permutazione figlio C

```
1  $cx_1 \leftarrow \text{random}(0, n - 1)$ ;  $cx_2 \leftarrow \text{random}(0, n - 1)$ ;  
2 if  $cx_1 > cx_2$  then  
3    $cx_1, cx_2 \leftarrow cx_2, cx_1$ ;  
4  $C \leftarrow$  array di  $n$  elementi;  $seg \leftarrow \{\}$ ;  
5 for  $i \leftarrow cx_1$  to  $cx_2$  do  
6    $C[i] \leftarrow P_1[i]$ ;  $seg.add(P_1[i])$ ;  
7  $j \leftarrow (cx_2 + 1) \bmod n$ ;  
8 per ogni  $i \in [cx_2 + 1, \dots, cx_2 + n]$  fai  
9    $pos \leftarrow i \bmod n$ ;  
10  if  $pos \notin [cx_1, cx_2]$  then  
11    while  $P_2[j] \in seg$  do  
12       $j \leftarrow (j + 1) \bmod n$ ;  
13     $C[pos] \leftarrow P_2[j]$ ;  $seg.add(P_2[j])$ ;  $j \leftarrow (j + 1) \bmod n$ ;  
14 return  $C$ ;
```

La **mutazione** ($p_m = 0.236$) applica casualmente: Swap (40%, scambia due elementi), Insert (30%, sposta un elemento), Inversion (30%, inverte un segmento).

3.3 Ricerca Locale Granulare

Quattro operatori sono applicati con probabilità $p_{ls} = 0.259$ in strategia **steepest descent**:

- **2-opt** e **Or-opt** (intra-route): eliminano incroci e riposizionano segmenti di 1–3 nodi;
- **Relocate** e **Exchange** (inter-route): spostano/scambiano clienti tra route diverse con filtro GLS [12] ($\gamma = 25$).

La Ricerca Locale Granulare limita le mosse inter-route ai soli γ vicini più prossimi, riducendo la complessità da $O(n^2)$ a $O(n \cdot \gamma)$. L'Algoritmo 3 illustra il funzionamento dell'operatore Relocate con filtro GLS in strategia steepest descent.

Algoritmo 3: Relocate con GLS (steepest descent)

Input: Rotte $\mathcal{R}$, distanze d , domande q , capacità Q , intorni Γ , depot 0, max iter m
Output: Rotte migliorate $\mathcal{R}^*$

```
1 loads[r]  $\leftarrow \sum_{n \in r} q[n] \forall r$ ;  
2 best  $\leftarrow \mathcal{R}$ ; bestCost  $\leftarrow \text{Cost}(\text{best})$ ; iter  $\leftarrow 0$ ;  
3 repeat  
4   iter  $\leftarrow \text{iter} + 1$ ; improved  $\leftarrow \text{false}$ ; bestMoveCost  $\leftarrow \text{bestCost}$ ;  
5   bestMove  $\leftarrow \text{null}$ ;  
6   per ogni route  $r_f$ , nodo  $v$  in pos. f di  $r_f$  fai  
7      $\delta_f \leftarrow d[\text{prev}(v), \text{next}(v)] - (d[\text{prev}(v), v] + d[v, \text{next}(v)])$ ;  
8     per ogni route  $r_t \neq r_f$  con loads[ $r_t$ ] +  $q[v] \leq Q$  fai  
9       per ogni pos.  $p \in [0, |r_t|]$  fai  
10        Filtro GLS: salta se entrambi i vicini non sono in  $\Gamma[v]$ ;  
11         $\delta_t \leftarrow d[pn, v] + d[v, nn] - d[pn, nn]$ ;  
12        if bestCost +  $\delta_f + \delta_t < \text{bestMoveCost}$  then  
13          bestMove  $\leftarrow (r_f, f, r_t, p)$ ;  
14    if bestMove  $\neq \text{null}$  then  
15      Applica la mossa; aggiorna loads e bestCost; improved  $\leftarrow \text{true}$ ;  
16 until  $\neg \text{improved} \vee (m > 0 \wedge \text{iter} \geq m)$ ;  
17 return best;
```

3.4 Ciclo Principale

L'Algoritmo 4 mostra il ciclo evolutivo. I migliori $e = 4$ individui sono preservati (elitismo); i restanti sono generati tramite selezione, OX, mutazione e ricerca locale. Al termine, i duplicati sono rimpiazzati con individui casuali. L'algoritmo termina dopo $FE = 350,000$ valutazioni.

Algoritmo 4: Ciclo Principale dell'HGA

Input: Istanza CVRP; parametri: $\mu, p_c, p_m, p_{ls}, k, e, \gamma, FE$
Output: Migliore soluzione S^*

```
1  $\mathcal{P} \leftarrow \{\text{SPLIT}(\text{NN}()), \text{SPLIT}(\text{CWS}())\}$ ;  
2 while  $|\mathcal{P}| < \mu$  do  
3    $\mathcal{P}.\text{append}(\text{SPLIT}(\text{RANDOM}()))$ ;  
4  $S^* \leftarrow \arg \min_{S \in \mathcal{P}} S.\text{cost}$ ; evals  $\leftarrow |\mathcal{P}|$ ;  
5 while evals  $< FE$  do  
6    $\mathcal{P}' \leftarrow \text{top-}e(\mathcal{P})$ ;  
7   while  $|\mathcal{P}'| < \mu$  do  
8     if random()  $< p_c$  then  
9        $\pi_c \leftarrow \text{OX}(\text{TOURN}(\mathcal{P}, k), \text{TOURN}(\mathcal{P}, k))$ ;  
10    else  
11       $\pi_c \leftarrow \text{flatten}(\text{TOURN}(\mathcal{P}, k).\text{routes})$ ;  
12    if random()  $< p_m$  then  
13       $\pi_c \leftarrow \text{MUTATE}(\pi_c)$ ;  
14     $S_c \leftarrow \text{SPLIT}(\pi_c)$ ;  $S_c \leftarrow \text{TWOOPT}(S_c)$ ;  
15    if random()  $< p_{ls}$  then  
16       $S_c \leftarrow \text{LOCALSEARCH}(S_c, \gamma)$ ;  
17     $\mathcal{P}'.\text{append}(S_c)$ ;  
18    if  $S_c.\text{cost} < S^*.\text{cost}$  then  
19       $S^* \leftarrow S_c$ ;  
20 Rimpiazza duplicati in  $\mathcal{P}'$ ;  
21  $\mathcal{P} \leftarrow \mathcal{P}'$ ;  
22 return  $S^*$ ;
```

3.5 Parametri Riepilogativi

Tabella 1: Parametri dell'HGA — Configurazione Tuned

Parametro	Valore
Popolazione (μ)	81
Crossover rate (p_c) / Mutation rate (p_m)	0.675 / 0.236
Local search rate (p_{ls})	0.259
Tournament (k) / Elite (e) / Granular (γ)	4 / 4 / 25
Max valutazioni (FE)	350,000

3.6 Architettura Software

L'algoritmo è integrato in un'architettura moderna con backend Python (FastAPI) [13] e frontend React con visualizzazione real-time tramite WebSocket. Il backend espone endpoint REST per avviare l'ottimizzazione e recuperare i risultati, mentre il frontend mostra la disposizione geografica delle route, i grafici di convergenza e le statistiche aggregate. La parallelizzazione multi-run sfrutta `ThreadPoolExecutor`, e l'accelerazione Numba JIT [14] gestisce i calcoli critici (matrici di distanza, Split DP).

4 Protocollo Sperimentale

4.1 Istanze

Gli esperimenti usano 10 istanze dal benchmark CVRPLIB [15], suddivise in quattro set con caratteristiche differenti (Set A: distribuzione uniforme; Set B: cluster geografici; Set E: distribuzione mista — il più difficile; Set P: domanda molto variabile).

Tabella 2: Istanze CVRPLIB utilizzate

Set	Istanza	Nodi	Veicoli	Ottimo
A	A-n45-k7	45	7	1,146
A	A-n60-k9	60	9	1,354
A	A-n80-k10	80	10	1,763
B	B-n56-k7	56	7	707
B	B-n66-k9	66	9	1,316
B	B-n78-k10	78	10	1,221
E	E-n76-k8	76	8	735
E	E-n101-k14	101	14	1,071
P	P-n50-k10	50	10	696
P	P-n101-k4	101	4	681

4.2 Setup Sperimentale

Per ogni istanza si eseguono $R = 5$ run indipendenti con semi diversi e budget $FE = 350,000$. Si misurano: costo migliore, deviazione standard, storico di convergenza e numero di veicoli utilizzati.

4.3 Configurazione Tuned

La configurazione `config_tuned` è ottenuta tramite tuning bayesiano con Optuna [16] su 100 trial, 4 istanze rappresentative (una per set), 3 run per trial, budget 100,000 FE. Lo spazio di ricerca include: $\mu \in [20, 150]$, $p_c \in [0.5, 1.0]$, $p_m \in [0.01, 0.5]$, $p_{ls} \in [0.01, 0.3]$, $k \in [2, 5]$, $e \in [0, 15]$, $\gamma \in [10, 40]$.

Il tuning ha prodotto: $\mu = 81$, $p_c = 0.675$, $p_m = 0.236$, $p_{ls} = 0.259$, $k = 4$, $e = 4$, $\gamma = 25$.

5 Risultati Sperimentali

I risultati sono ottenuti con la configurazione `config_tuned` ($\mu = 81$, $p_c = 0.675$, $p_m = 0.236$, $p_{ls} = 0.259$, $\gamma = 25$, $FE = 350,000$, $R = 5$) ottimizzata tramite Optuna.

5.1 Risultati della Configurazione Tuned

Tabella 3: Risultati HGA — Configurazione Tuned ($\mu = 81$, $k = 4$, $e = 4$, $\gamma = 25$, $FE = 350,000$, $R = 5$)

Istanza	Best	Mean	Std	Ottimo	Gap%	Vei.
A-n45-k7	1146.77	1146.77	0.00	1,146	0.07%	7
A-n60-k9	1355.80	1355.80	0.00	1,354	0.13%	9
A-n80-k10	1784.41	1785.00	0.74	1,763	1.21%	10
B-n56-k7	712.92	712.92	0.00	707	0.84%	7
B-n66-k9	1326.20	1326.56	0.44	1,316	0.78%	9
B-n78-k10	1227.90	1228.45	0.98	1,221	0.56%	10
E-n76-k8	740.66	741.29	1.27	735	0.77%	8
E-n101-k14	1090.72	1092.38	0.83	1,071	1.84%	14
P-n50-k10	700.66	701.37	1.42	696	0.67%	10
P-n101-k4	691.29	691.83	0.66	681	1.51%	4

I risultati mostrano un gap medio dello 0.84%, con picchi di eccellenza su A-n45-k7 (gap 0.07%) e A-n60-k9 (gap 0.13%). La deviazione standard molto contenuta (spesso $< 1\%$ della media) conferma la robustezza dell'algoritmo. L'istanza più difficile si conferma E-n101-k14 (gap 1.84%) a causa dell'elevato numero di clienti combinato con un'alta densità di veicoli.

5.2 Convergenza e Rotte

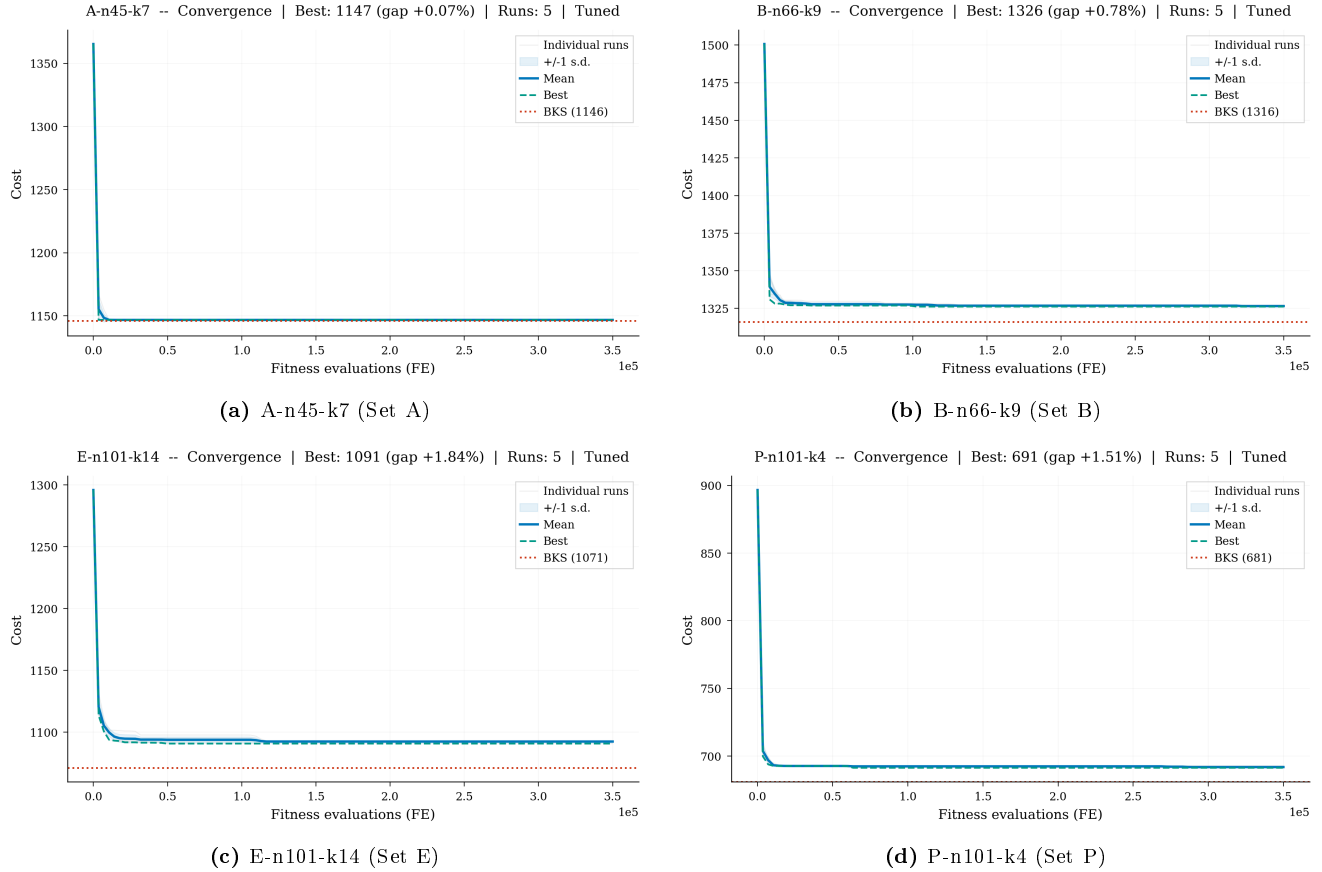
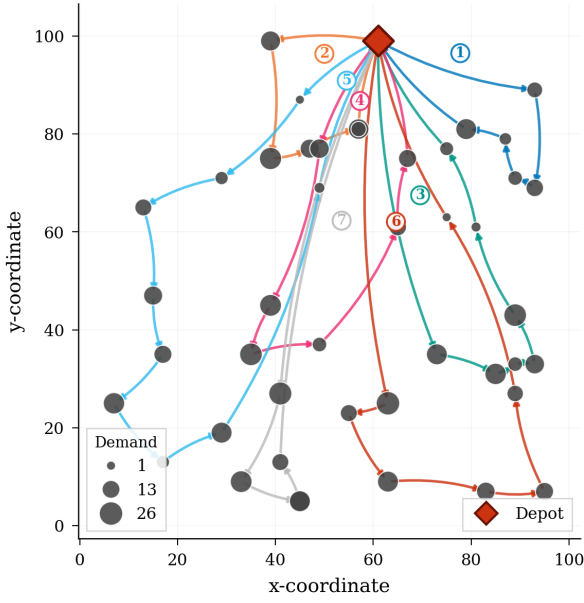


Figura 1: Curve di convergenza per quattro istanze rappresentative — Configurazione Tuned.

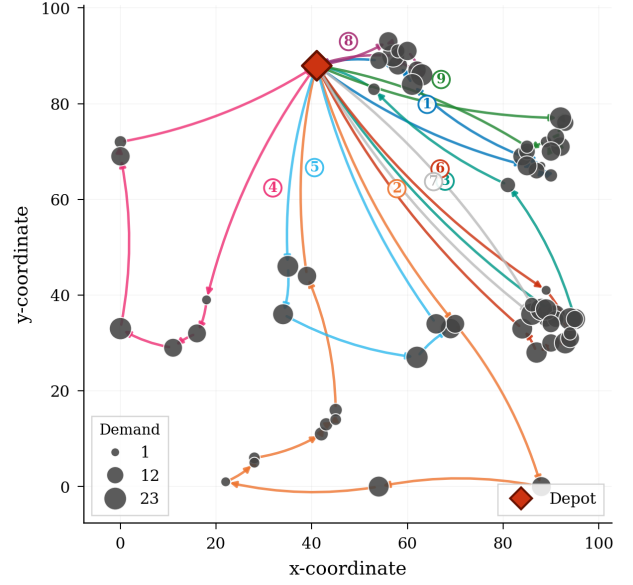
Le curve mostrano un rapido miglioramento nelle prime 50,000–100,000 valutazioni, seguito da un plateau di raffinamento graduale. La convergenza è più rapida per le istanze di set A e B, mentre per E-n101-k14 e P-n101-k4 l'algoritmo continua a migliorare fino alle fasi finali.

A-n45-k7 -- Best solution | Tuned
Cost: 1147 (gap +0.07%) Vehicles: 7/7 Customers: 44



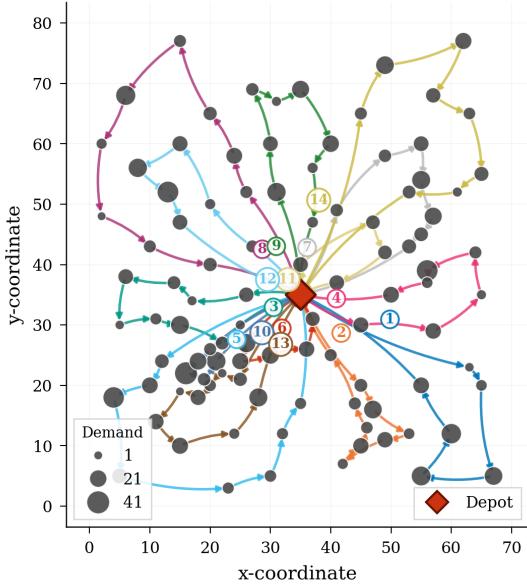
(a) Rotte per A-n45-k7

B-n66-k9 -- Best solution | Tuned
Cost: 1326 (gap +0.78%) Vehicles: 9/9 Customers: 65



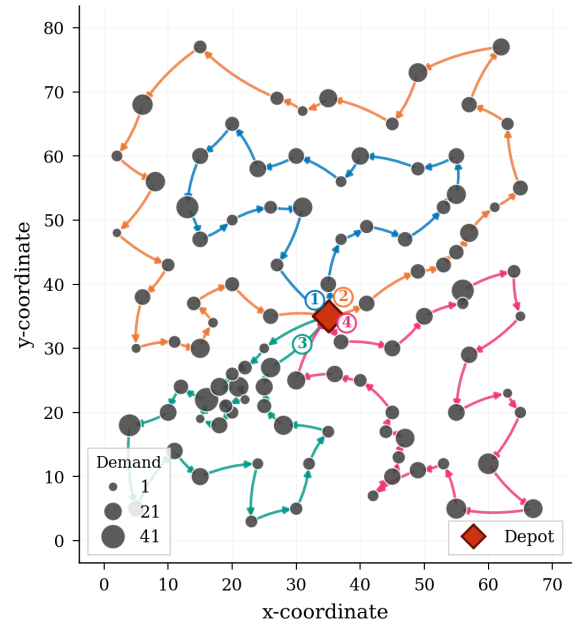
(b) Rotte per B-n66-k9

E-n101-k14 -- Best solution | Tuned
Cost: 1091 (gap +1.84%) Vehicles: 14/14 Customers: 100



(c) Rotte per E-n101-k14

P-n101-k4 -- Best solution | Tuned
Cost: 691 (gap +1.51%) Vehicles: 4/4 Customers: 100

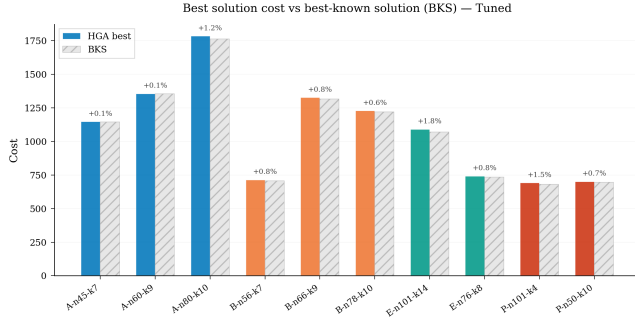


(d) Rotte per P-n101-k4

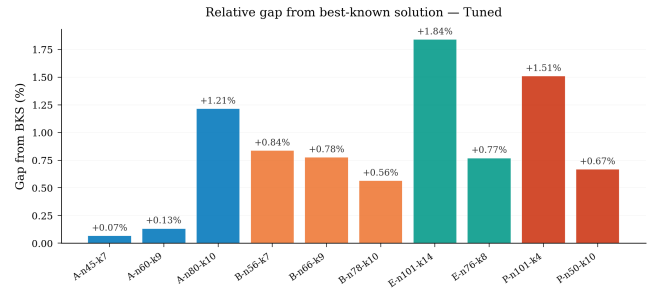
Figura 2: Visualizzazione grafica delle rotte finali — Configurazione Tuned.

Le rotte appaiono logicamente coerenti: i clienti sono raggruppati in cluster geograficamente compatti, senza incroci evidenti tra i percorsi. L'istanza P-n101-k4 (solo 4 veicoli per 100 clienti) produce rotte molto lunghe ma geometricamente ben strutturate.

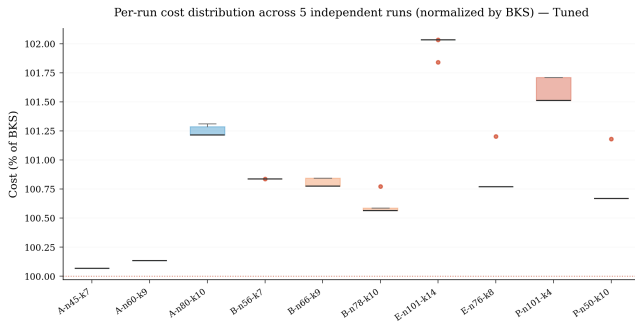
5.3 Analisi Statistica



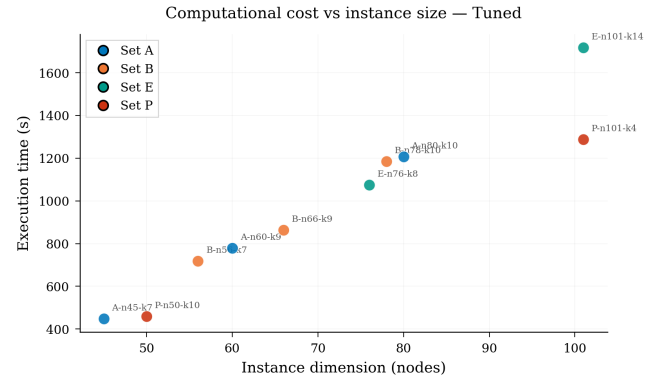
(a) Costi HGA vs BKS



(b) Gap percentuale dal BKS



(c) Distribuzione costi normalizzati



(d) Tempo di calcolo vs Dimensione

Figura 3: Statistiche globali sulle prestazioni e la stabilità — Configurazione Tuned.

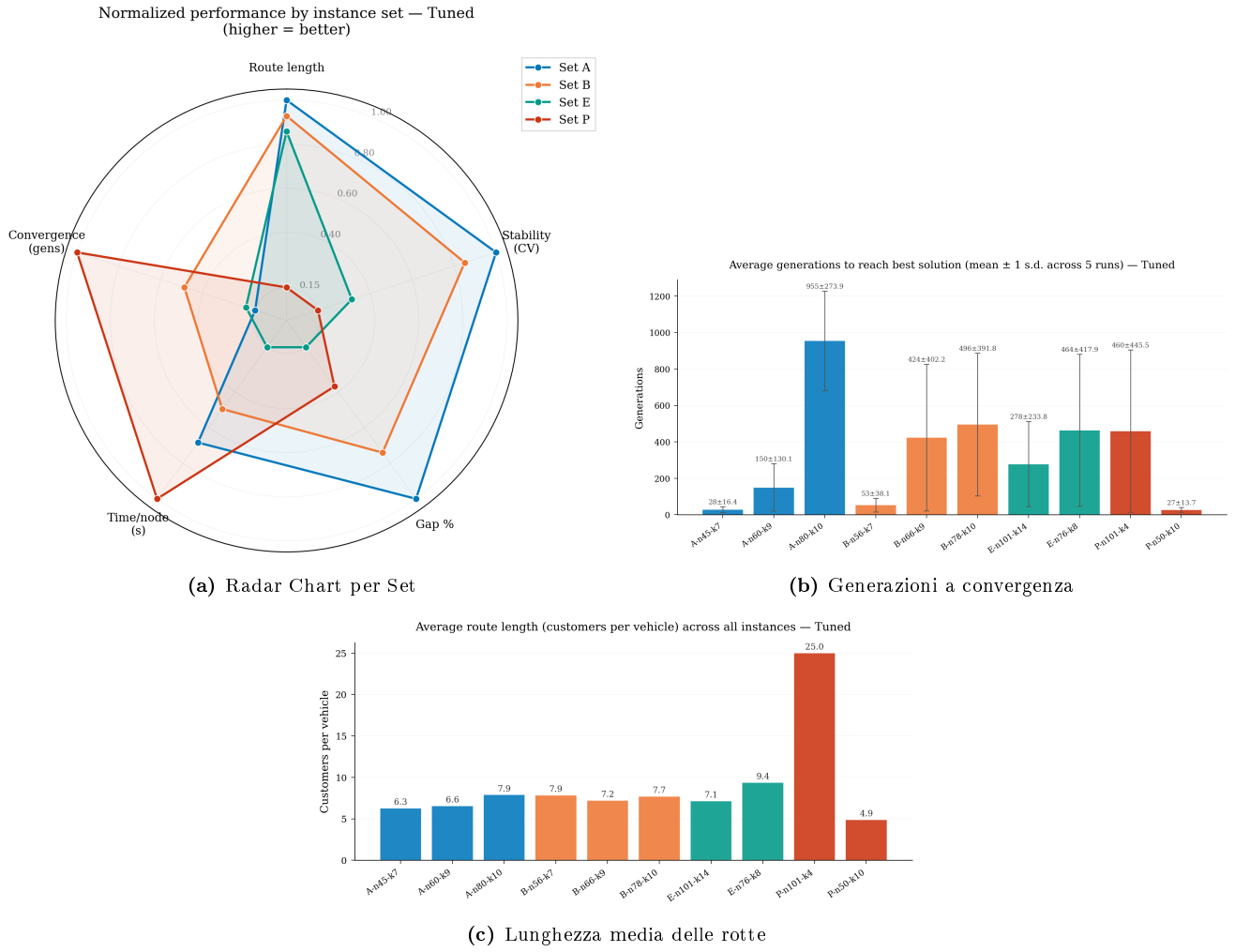


Figura 4: Prestazioni aggregate, convergenza evolutiva e struttura delle rotte.

5.4 Confronto tra Varianti di Configurazione

Per valutare l'impatto dei parametri, la configurazione Tuned è stata confrontata con sei varianti alternative, riassunte in Tabella 4.

Tabella 4: Varianti di configurazione a confronto

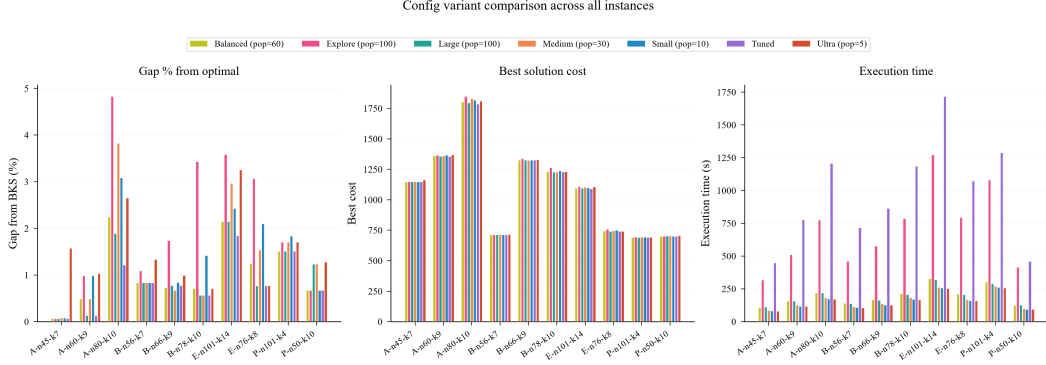
Variante	μ	k	e	γ	Generazioni
Tuned	81	4	4	25	$\sim 4,300$
Balanced	60	3	4	12	$\sim 5,800$
Large	100	4	5	15	$\sim 3,500$
Medium	30	3	3	7	$\sim 11,600$
Small	10	2	2	3	$\sim 35,000$
Fast	5	2	1	2	$\sim 70,000$

La Tabella 5 riporta il confronto completo su tutte le 10 istanze.

Tabella 5: Confronto tra varianti di configurazione — costo della soluzione migliore e gap dal BKS

Instance	Optimal	Tuned	Balanced	Explore	Large	Medium	Small	Fast
A-n45-k7	1,146	1,146.77 (+0.07%)	1,146.77 (+0.07%)	1,146.77 (+0.07%)	1,146.77 (+0.07%)	1,146.91 (+0.08%)	1,146.91 (+0.08%)	1,164.09 (+1.58%)
A-n60-k9	1,354	1,355.80 (+0.13%)	1,360.59 (+0.49%)	1,367.35 (+0.99%)	1,355.80 (+0.13%)	1,360.59 (+0.49%)	1,367.34 (+0.99%)	1,367.97 (+1.03%)
A-n80-k10	1,763	1,784.41 (+1.21%)	1,802.63 (+2.25%)	1,848.12 (+4.83%)	1,796.37 (+1.89%)	1,830.48 (+3.83%)	1,817.37 (+3.08%)	1,809.79 (+2.65%)
B-n56-k7	707	712.92 (+0.84%)	712.92 (+0.84%)	714.69 (+1.09%)	712.92 (+0.84%)	712.92 (+0.84%)	712.92 (+0.84%)	716.42 (+1.33%)
B-n66-k9	1,316	1,326.20 (+0.78%)	1,325.57 (+0.73%)	1,338.88 (+1.74%)	1,326.20 (+0.78%)	1,324.84 (+0.67%)	1,327.10 (+0.84%)	1,329.01 (+0.99%)
B-n78-k10	1,221	1,227.90 (+0.56%)	1,229.69 (+0.71%)	1,262.89 (+3.43%)	1,227.90 (+0.56%)	1,227.90 (+0.56%)	1,238.33 (+1.42%)	1,229.69 (+0.71%)
E-n101-k14	1,071	1,090.72 (+1.84%)	1,093.96 (+2.14%)	1,109.37 (+3.58%)	1,093.96 (+2.14%)	1,102.78 (+2.97%)	1,097.02 (+2.43%)	1,105.88 (+3.26%)
E-n76-k8	735	740.66 (+0.77%)	744.18 (+1.23%)	757.56 (+3.07%)	740.66 (+0.77%)	746.28 (+1.53%)	750.44 (+2.10%)	740.66 (+0.77%)
P-n101-k4	681	691.29 (+1.51%)	691.29 (+1.51%)	692.64 (+1.71%)	691.29 (+1.51%)	692.64 (+1.71%)	693.54 (+1.84%)	692.64 (+1.71%)
P-n50-k10	696	700.66 (+0.67%)	700.66 (+0.67%)	700.66 (+0.67%)	704.61 (+1.24%)	704.61 (+1.24%)	700.66 (+0.67%)	704.91 (+1.28%)

Tuned: pop=81, tournament=4, elite=4, granular=25; **Balanced:** pop=60, tournament=3, elite=4, granular=12; **Explore:** pop=100, tournament=2, elite=1, granular=15; **Large:** pop=100, tournament=4, elite=5, granular=15; **Medium:** pop=30, tournament=3, elite=3, granular=7; **Small:** pop=10, tournament=2, elite=2, granular=3; **Fast:** pop=5, tournament=2, elite=1, granular=2


Figure 5: Confronto visivo tra le sette varianti di configurazione: gap %, costo migliore e tempo di esecuzione.

5.5 Discussione

L'analisi dei risultati permette di trarre diverse osservazioni:

- **Qualità:** La configurazione Tuned ($\mu = 81$) ottiene un gap medio dello 0.84%. Rispetto a Large ($\mu = 100$, gap $\sim 1.0\%$), Tuned raggiunge prestazioni superiori con una popolazione più piccola, grazie ai parametri calibrati da Optuna ($p_c = 0.675$ anziché 0.8, $\gamma = 25$ anziché 15).
- **Effetto della popolazione:** Riducendo μ da 100 (Large) a 5 (Fast), il gap medio peggiora progressivamente (Large: $\sim 1.0\%$, Fast: $\sim 1.7\%$). Tuned e Balanced, con popolazioni intermedie (81 e 60), rappresentano il miglior compromesso tra diversità genetica e velocità.
- **Robustezza:** La deviazione standard è generalmente contenuta in tutte le configurazioni ($\sigma < 1$ su 7/10 istanze), indicando stabilità intrinseca dell'algoritmo.
- **Trade-off qualità/tempo:** Le configurazioni con popolazione minore richiedono più generazioni ma minor tempo per generazione. Il tempo totale è dominato dal budget fisso di FE (350,000), rendendo i tempi comparabili tra tutte le varianti.

6 Conclusioni

Il progetto ha dimostrato l'efficacia dell'Hybrid Genetic Algorithm per il CVRP, producendo soluzioni di alta qualità (gap medio 0.84%) sulle istanze benchmark CVRPLIB. L'approccio ibrido, che combina la potenza esplorativa degli algoritmi genetici con l'efficacia della ricerca locale, si è dimostrato particolarmente efficace.

6.1 Scelte Progettuali

- **Split di Prins [4]:** l'uso dello split come decodifica è fondamentale — invece di codificare esplicitamente le route, la DP trova la partizione ottimale, riducendo lo spazio di ricerca e garantendo l'ottimalità della decodifica data la permutazione.
- **Ricerca Locale Granulare:** l'integrazione della GLS con $\gamma = 25$ permette di esplorare un vicinato inter-route molto più ampio rispetto alle configurazioni standard ($\gamma = 15$), senza il costo computazionale della ricerca esaustiva $O(n^2)$.
- **Tuning con Optuna [16]:** ha scoperto una configurazione non ovvia ($\mu = 81$, $p_c = 0.675$, $p_m = 0.236$, $p_{ls} = 0.259$) che supera le configurazioni progettate manualmente.
- **Numba JIT [14]:** performance C/C++ in Python puro per i calcoli critici (matrici di distanza, Split DP, operatori locali).
- **Architettura client-server:** FastAPI + React con visualizzazione real-time via WebSocket [13].

6.2 Difficoltà e Risultati

Le sfide principali hanno riguardato la gestione dei vincoli di capacità durante crossover e mutazione (risolta elegantemente dallo Split) e il bilanciamento esplorazione-sfruttamento (ottimizzato da Optuna con $p_{ls} = 0.259$). Su istanze con 101 nodi, lo Split $O(n^2)$ e la ricerca locale diventano computazionalmente significativi, richiedendo l'uso di Numba JIT per mantenere tempi accettabili.

I punti di forza dell'implementazione includono: robustezza e consistenza tra run ($\sigma < 1$ su 7/10 istanze), gap medio 0.84%, eccellenza sui set A e B (gap $< 0.15\%$ su A-n45-k7 e A-n60-k9), e un'architettura software moderna con visualizzazione real-time.

6.3 Sviluppi Futuri

Possibili estensioni del lavoro includono:

- Implementazione di una ricerca locale più aggressiva (Variable Neighborhood Search) [5];
- Tecniche di diversificazione della popolazione (crowding, fitness sharing);
- Estensione ad altre varianti del VRP (VRPTW con finestre temporali, VRPPD con pickup and delivery);
- Implementazione di un algoritmo memetico con ricombinazione ottimale dei percorsi (edge assembly crossover) [17];
- Integrazione con un solver esatto per il post-processing delle route (set partitioning).

Bibliografia

- [1] George B. Dantzig e John H. Ramser. «The Truck Dispatching Problem». In: *Management Science* 6.1 (1959), pp. 80–91. DOI: 10.1287/mnsc.6.1.80.
- [2] Paolo Toth e Daniele Vigo, cur. *The Vehicle Routing Problem*. Society for Industrial e Applied Mathematics (SIAM), 2002. ISBN: 978-0-89871-579-8.
- [3] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989. ISBN: 978-0-201-15767-3.
- [4] Christian Prins. «A simple and effective evolutionary algorithm for the vehicle routing problem». In: *Computers & Operations Research* 31.12 (2004), pp. 1985–2002. DOI: 10.1016/S0305-0548(03)00158-8.
- [5] Thibaut Vidal et al. «A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems». In: *Operations Research* 60.3 (2012), pp. 611–624. DOI: 10.1287/opre.1120.1048.
- [6] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975. ISBN: 978-0-262-58111-0.
- [7] Shen Lin e Brian W. Kernighan. «An Effective Heuristic Algorithm for the Traveling-Salesman Problem». In: *Operations Research* 21.2 (1973), pp. 498–516. DOI: 10.1287/opre.21.2.498.
- [8] İlhan Or. «Traveling Salesman-Type Combinatorial Problems and their Relation to the Logistics of Regional Blood Banking». Tesi di dott. Northwestern University, 1976.
- [9] Geoff Clarke e John W. Wright. «Scheduling of Vehicles from a Central Depot to a Number of Delivery Points». In: *Operations Research* 12.4 (1964), pp. 568–581. DOI: 10.1287/opre.12.4.568.
- [10] David E. Goldberg e Kalyanmoy Deb. «A Comparative Analysis of Selection Schemes Used in Genetic Algorithms». In: *Foundations of Genetic Algorithms*. Vol. 1. Morgan Kaufmann, 1991, pp. 69–93.
- [11] Lawrence Davis. «Applying Adaptive Algorithms to Epistatic Domains». In: *Proceedings of the 9th International Joint Conference on Artificial Intelligence (IJCAI)*. 1985, pp. 162–164.
- [12] Paolo Toth e Daniele Vigo. «The Granular Tabu Search and Its Application to the Vehicle-Routing Problem». In: *INFORMS Journal on Computing* 15.4 (2003), pp. 333–346. DOI: 10.1287/ijoc.15.4.333.24890.
- [13] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. Accessed: 2025-06-01.
- [14] Siu Kwan Lam, Antoine Pitrou e Stanley Seibert. «Numba: A LLVM-Based Python JIT Compiler». In: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*. 2015, pp. 1–6. DOI: 10.1145/2833157.2833162.
- [15] CVRPLIB. *Capacitated Vehicle Routing Problem Library*. <http://vrp.atd-lab.inf.puc-rio.br/>. Accessed: 2025-06-01.
- [16] Takuya Akiba et al. «Optuna: A Next-Generation Hyperparameter Optimization Framework». In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. 2019, pp. 2623–2631. DOI: 10.1145/3292500.3330701.
- [17] Thibaut Vidal et al. «A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows». In: *Computers & Operations Research* 40.1 (2013), pp. 475–489. DOI: 10.1016/j.cor.2012.07.018.